

Practical 3 — MapReduce on Forest Fire Dataset

Cell 1 — Download Dataset

```
import urllib.request
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/forest-fires/forestfires.csv"
urllib.request.urlretrieve(url, r'C:\Users\DELL\Downloads\forestfires.csv')
print("Downloaded!")
```

Cell 2 — Load Dataset

```
import pandas as pd
df = pd.read_csv(r'C:\Users\DELL\Downloads\forestfires.csv')
print(df.shape)
df.head()
```

Cell 3 — MAP Step

```
def mapper(df):
    mapped = []
    for _, row in df.iterrows():
        key = row['month']
        value = row['area']
        mapped.append((key, value))
    return mapped

mapped_data = mapper(df)
print("Sample mapped data:")
print(mapped_data[:5])
```

Cell 4 — SHUFFLE Step

```
from collections import defaultdict

def shuffle(mapped_data):
    shuffled = defaultdict(list)
    for key, value in mapped_data:
        shuffled[key].append(value)
    return shuffled

shuffled_data = shuffle(mapped_data)
for k, v in list(shuffled_data.items())[:3]:
    print(f"Month: {k}, Areas: {v[:3]}")
```

Cell 5 — REDUCE Step

```
def reducer(shuffled_data):
    result = {}
    for month, areas in shuffled_data.items():
        result[month] = {
            'total_area': sum(areas),
            'count': len(areas),
            'avg_area': round(sum(areas)/len(areas), 2)
        }
    return result

result = reducer(shuffled_data)
```

```
for month, stats in result.items():  
    print(f"Month: {month} => {stats}")
```

Cell 6 — Show as DataFrame

```
result_df = pd.DataFrame(result).T  
result_df = result_df.sort_values('total_area', ascending=False)  
print(result_df)
```